MDMC
Master in Data Management and Curation

AREA SCIENCE PARK

SISSA

Python II: Object-Oriented Programming

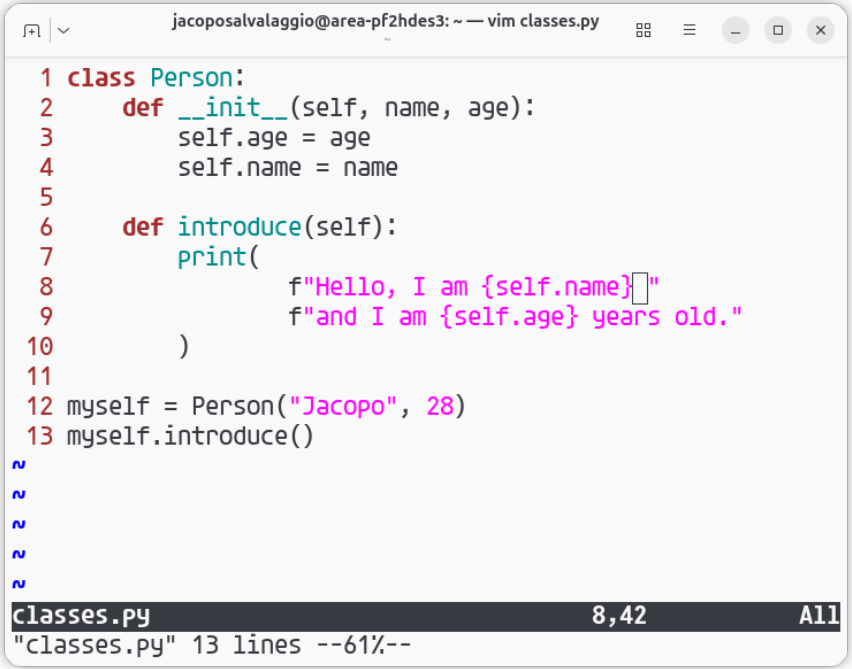
Jacopo Salvalaggio



Image credit: vecteezy.com

CC BY

Goals



Objectives of the lecture:

* Learn how to create **classes** in Python
* Understand class **properties** and **inheritance**
* Master many useful **special methods**

Objects

An **object** is any data with **state** and defined **behavior**. In Python, **everything* is an object**.

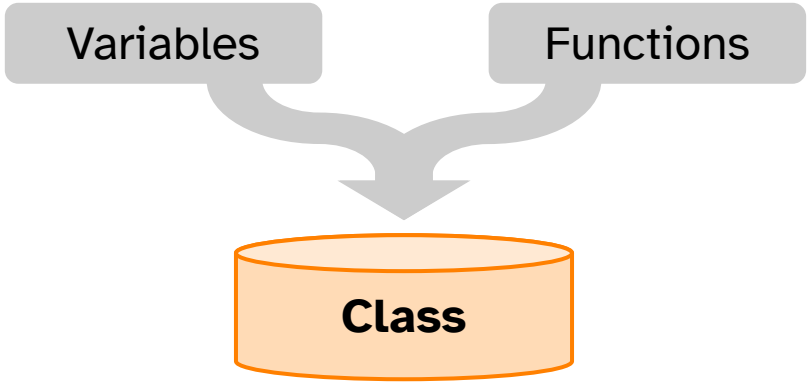
Example: objects in python

```
>>> a = 1 # a's value represents its state
>>> a * 2 # a can be multiplied (behavior)
>>> print(a) # print has a specific behavior
>>> print.__doc__ # print (a function) also has state
```

Classes and objects

Classes define special types of **objects**.

You can see them as a way of **packaging variables and functions** into a single unit.



Classes

According to the official glossary, a **class** is a "**template for creating user-defined objects**".

When writing a class, the user specifies its **state** (**data attributes**) and **behavior** (**methods**)

Example: a simple class

```
class Test:
    state = "Some state"

    def behavior(self):
        print("Behaving some way!")
```

Classes and instances

Classes are templates (**types** if you will) for objects, **class instances are actual objects** existing in the code.

So, **an instance of a class is an object that has the properties defined by the class**.

The **`self`** variable **represents the name of each instance**.

Example: instances

```
>>> class Test: ... # Defining the class
>>> x = Test() # Assigning x to be an instance of the
Test class
```

The dot operator

Objects stored within the class are retrieved with the **dot (`.`) operator**. In the previous example:

* `x.state` retrieves the attribute "state" (also `Test.state` would have worked here)
* `x.behavior()` retrieves the method "behavior": notice how **we do not pass any value for `self`** since it is set to the name of the instance, as if we were calling `Test.behavior(x)`

Initializing

`__init__` is a **special method** that is executed to **initialize** the class. **Its arguments are passed when instantiating** (apart from `self` which is omitted):

- If no `__init__` is provided* or its only required argument is `self`, we create a new instance by simply `x = Test()`
- Otherwise, we pass the arguments to the class name as in `y = SomeInitClass(arg1, arg2, ...)`

* Some implementation of `__init__` is **always present**, even when you do not define it.

A closer look: the class

Example: another simple class

```
class Person:
    species = "Human"
    def __init__(self, name, age):
        self.age = age
        self.name = name

    def introduce(self):
        print(f"I am {self.name}
and I am {self.age} years old.")
```

Class attribute*

`self` represents the name of the instance (not the class)

Methods*

Instance attribute*

A closer look: the instance

Example: attributes and instantiating

```
>>> Person.species # OK, class attribute
'Human'
>>> Person.age # Not OK, instance attribute
[...] AttributeError: type object 'Person' has no
attribute 'age'
>>> myself = Person("Jacopo", 28)
>>> myself.species, myself.age # Both are OK here
('Human', 28)
>>> myself.introduce()
I am Jacopo and I am 28 years old.
```

Inheritance

Classes can **inherit** the attributes and methods of other classes, with the idea to **extend** them

Example: parent class

```
class Mammal(Animal):
    is_hairy = True
```

Example: grandparent class

```
class Animal:
    def __init__(self, name):
        self.name = name
```

Example: child class

```
class Dog(Mammal):
    def woof(self):
        print("Woof!")
```

Overriding

It is possible to **override** a method or attribute after inheriting it, usually to **extend its functionality**. The original inherited method can be referred by the explicit name of the original class or with the **`super()`** function.

Example: parent class

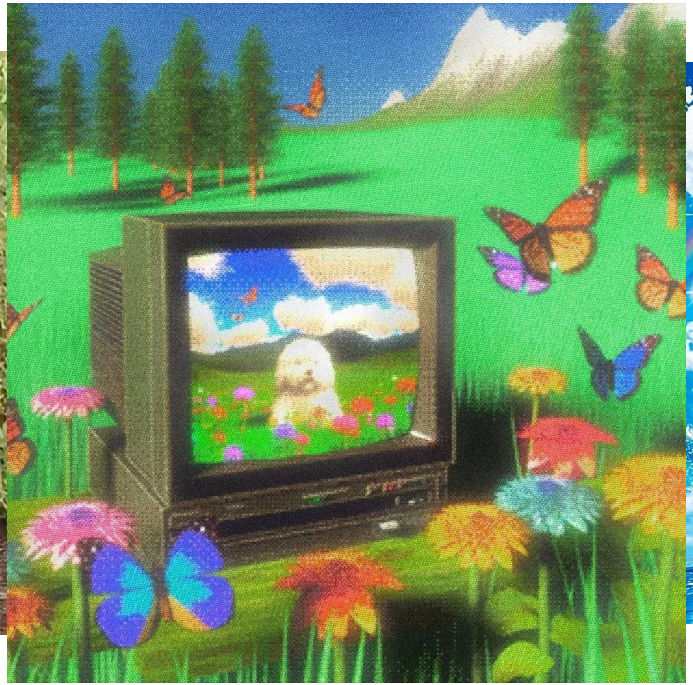
```
class Animal:
    def __init__(self, name):
        self.name = name
    def info(self):
        print(f"I am an animal
named {self.name}")
```

Example: child class

```
class Dog(Animal):
    def __init__(self, name):
        super().__init__(name)
        print("Woof!")
    def info(self):
        Animal.info(self)
        print("I am a dog.")
```

Time







What's the point?

You can write Python without every writing a class (even though you would still be using them all the time).

In fact, in a script or notebook it would be arguably best not to define any...

However, in structured projects they are a **very powerful tool to group functionality**.

Also, they allow you to **customize** the behavior of **basic operations** with **special methods**.

Special methods

Special ("**dunder**") methods allow to **customize** the **basic behavior** of python for a given class.

They are **not** generally **called directly**, but are executed when running other operations on the class instance (e.g. initializing, accessing an attribute, summing, ...)

Example: `__init__`

`__init__`

Runs when initializing the class, allows full customization of what happens when we instantiate an object.

Example: weird checks

```
class Weird:
    def __init__(self):
        print("Initializing Weird type object...")
        if "weirdness" in globals() and weirdness != 0:
            print("Found weirdness.")
        else:
            raise Exception("No weirdness found :(")
```

`__str__`

Defines string representation of the instance and is invoked when running `str(object)` (or `print(object)`).

Example: printing a string

```
class Person:
    def __init__(self, name, surname):
        self.name, self.surname = name, surname
    def __str__(self):
        return f"{self.name} {self.surname}"
```

Comparison

You can define the **result of a comparison** (=, <, ...) between objects of the same class with `__eq__`, `__gt__`, ...

Example: coordinates

```
class Coordinates:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __eq__(self, other):
        if self.x == other.x and self.y == other.y:
            return True
        else:
            return False
    def __gt__(self, other):
        if self.x**2 + self.y**2 > other.x**2 +
other.y**2:
            return True
        else:
            return False
```

`__setattr__`

Runs when setting an attribute, allows you to hijack what happens every time you run `object.attr = ...`.

Example: forbidden words

```
class Censored:
    def __init__(self, word, naughty=False):
        self.naughty = naughty
        self.word = word
    def __setattr__(self, name, value):
        if value in ("mean", "ugly") and not self.naughty:
            raise ValueError("No setting bad words!")
        super().__setattr__(name, value)
```

`__getattribute__`

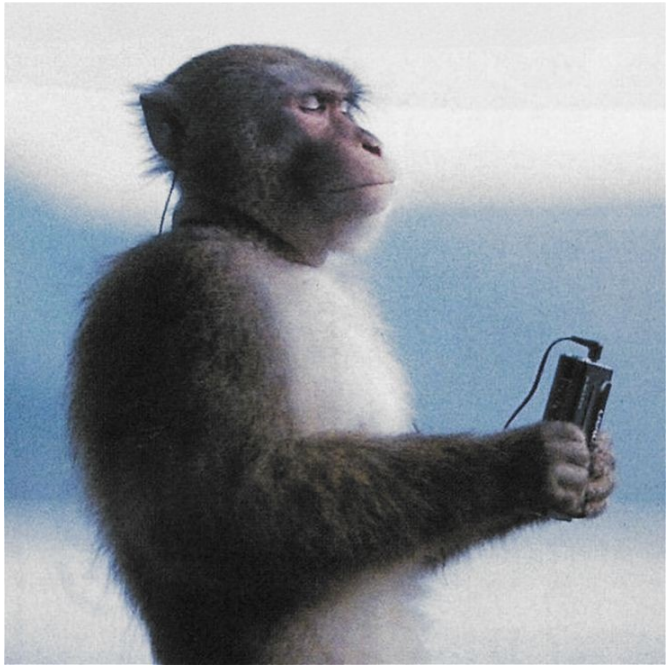
Runs when getting an attribute*, the other side of the coin to `__setattr__`

Example: forbidden words pt. 2

```
class CensoredGet(Censored):
    def __getattribute__(self, name):
        value = super().__getattribute__(name)
        if value in ("mean", "ugly"):
            print("Naughty naughty...")
        return value
```

* `__getattr__` is also a thing but let's not get into that...

More time (almost there)





Tangent: annotations

Newer versions of Python allow to **annotate types** of variables.
Still, no type enforcement is done by the interpreter.
Great for readability :)

Example: annotations

```
class Book:
    material: str = "paper"
    font: str = "Palatino"
    def count_pages(self) -> int:
        number: int = 100
        return number
```

Tangent: decorators

Decorators are **functions returning** other **functions**.

There are also **decorators** made **for classes**.

Example: basic decorator

```
def my_decorator(func):
    def auxiliary_function():
        print("🎀")
        func()
    return auxiliary_function

@my_decorator
def hello_world():
    print("Hello world!")
```

Properties

A **property** is a **method** that accepts no arguments and can be **accessed as if it were an attribute**. Useful to define quantities that can change dynamically.

Example: a property

```
class SomeNumbers:
    my_number = 1
    @property # yes, a decorator
    def double_my_number(self):
        return self.my_number * 2
```

Example: using a property

```
>>> x = SomeNumbers()
>>> x.my_number = 3
>>> x.double_my_number
6
```

Static methods

A static method is **not implicitly passed `self`** as a variable.
Essentially it is just a normal function in a class.

Example: static methods

```
class Static:
    desc = "I ran out of ideas"
    @staticmethod
    def double(number):
        return number * 2
```

Class methods

A class method's **first** (implict) **argument is the class** `cls`, not the instance `self`. Handy when inheriting.

Example: class methods

```
class FileReader:
    @classmethod
    def batch_init(cls, list_of_files):
        class_list = []
        for file in list_of_files:
            class_instance = cls(file)
            class_list.append(class_instance)
        return class_list
```

No need to hard-code class name!

"Private" attributes

Purely as a **convention**, names of attributes and methods that are "**private**", i.e. should not be accessed by the final user*, are made to start with an **underscore**, "_".

Example: a "private" method

```
class Watch:
    def _get_time(self): ...
    def _display_watch(self, time): ...
    def time(): # User should call this!
        current_time = self._get_time()
        self._display_watch(current_time)
```

Abstract classes

Abstract classes provide a **blueprint** for other classes to be inherited. Any **inherited abstract method must be defined** by the inheriting class.

Example: an abstract class

```
from abc import ABC
class CustomSetter(ABC):
    @abstractmethod
    def __setattr__(self, name, value):
        pass
```

Any class inheriting from `CustomSetter` **must*** define a `__setattr__` method.

Why...?

Abstract classes provide a conceptual way to define a **fundamental type** of object.

You can then have different classes inheriting the basic structure. I know, it is indeed very abstract.

Example: why defining an abstract class

```
from abc import ABC
class CustomSetter(ABC): ...
class OnlyIntAttributes(CustomSetter): ...
class AddOneEveryTimeYouSet(CustomSetter): ...
```

AREA SCIENCE PARK

SISSA

OGS Istituto Nazionale di Oceanografia e di Geofisica Sperimentale

ARCTUR

REGIONE AUTONOMA FRIULI VENEZIA GIULIA

E-ARGO



Image credit: Abel+Imray

Thank you for your attention :)

Image credits for background rainbow gradient: Image Colorful Rainbow Pattern by sgagnonou812 at Wallpapers.com